

Report TWEB

Studenti: Mattia Nada(1043526), Fiorella Beqiraj (1076797)

Corso: Tecnologie Web 2024-2025

Introduzione

Questo progetto mostra come abbiamo esplorato il mondo del cinema costruendo uno strumento che permetta di navigarlo e raccontarlo. Abbiamo dato vita a **Movie APP**, una *Single Page Application* nata per riordinare il vasto catalogo cinematografico ed aiutare gli utenti nella scoperta di nuovi film o vecchi cult attraverso recensioni e discussioni in tempo reale.

Architettura del sistema

Il nostro lavoro si è sviluppato seguendo un approccio a **Microservizi**, una scelta che ci ha permesso di separare la gestione dei dati in **statici** e **dinamici**, in un sistema composto da tre componenti principali (Server) che comunicano tra loro tramite il protocollo HTTP (REST API)

1. Frontend Server

Gestisce l'interfaccia utente e gli asset statici (CSS, JS) e agisce da API gateway (dispatcher) : il client interroga questo server che a sua volta inoltra le richieste ai servizi Java e Mongo per poi gestire le loro risposte e restituirle al frontend. Tutto ciò usando tecnologie Node.js, Express, Handlebars.

2. Backend Node.js-Express

Gestisce i dati degli utenti (mutabili, salvati su un database MongoDB) che hanno bisogno di essere aggiornati spesso, recupera le recensioni e permette il salvataggio e recupero delle chat room e messaggi degli utenti

3. Backend SpringBoot Server

Gestisce i dati statici dei film (immutabili, salvati su un DataBase PostgreSQL). Garantisce che i dati abbiano una relazione strutturali consistenti, fornisce le informazioni anagrafiche dei film e gestisce le relazioni complesse tramite jpa/hibernate.

Funzionalità

Solution: Nel progettare l'interfaccia ci siamo posti come rendere visualizzabile un catalogo vasto senza sacrificare le performance. La soluzione per mantenere la single page application (utilizzando Vanilla JavaScript e AJAX) è quella di avere una Pageable query, la quale invece di farci scaricare l'intero dataset ci permette di scaricare i dati in chunk di 30 film. Questo ci permette di continuare la navigazione in maniera fluida lasciando che l'interfaccia lanci chiamate asincrone mirate al momento in cui ci avviciniamo a raggiungere la fine della pagina.

Issues: La sfida principale consisteva nel presentare un vasto dataset di film senza sovraccaricare il browser dell'utente o il server. Caricare migliaia di record in una volta sola causa tempi di avvio lenti e un elevato consumo di memoria. Inoltre, la navigazione tradizionale multi-pagina risulta obsoleta.

Funzionamento: Quando l'utente scorre o richiede dati, il frontend invia una richiesta con parametri specifici. Il Controller Java (utilizzando Spring Data JPA) recupera solo quella specifica "fetta" (page) del database.

Requisiti: Questo design soddisfa pienamente il requisito di recupero efficiente dei dati e di interfacce utente reattive. Usando la comunicazione asincrona (AJAX) e la progettazione API REST, garantiamo il disaccoppiamento tra frontend e backend come richiesto.

Limitazioni: Seppur sia possibile il filtraggio sui generi, ci sarebbero la possibilità di aggiungere più filtri. Inoltre utilizzando query dinamiche si potrebbero utilizzare più filtri contemporaneamente senza la necessità di definire manualmente query statiche per ogni combinazione di filtri.

2. Query di ricerca tramite Search

Issues: L'uso di operatori standard (come LIKE) su grandi dataset è inefficiente e non garantisce un ordinamento intelligente: un utente che cerca un titolo esatto si aspetta di vederlo come primo risultato, non mescolato tra sequel.

Soluzione: Abbiamo bypassato l'ORM standard implementando una Native Query PostgreSQL ottimizzata. Usiamo `to_tsvector` combinato con `REPLACE` per trasformare l'input utente in una ricerca logica "AND" (tutte le parole devono essere presenti).

Un ranking a 3 Livelli: Un costrutto CASE definisce la gerarchia visuale:

1. Match Esatto: Priorità massima.
2. Prefisso: Il titolo inizia con la parola cercata.
3. Contesto: Il termine è presente nel titolo.

Ottimizzazione: A parità di rilevanza, ordiniamo per rating e data, recuperando i poster con una singola JOIN e limitando l'output a 20 risultati per garantire latenza zero.

Requisiti: Soddisfa i requisiti di efficienza e ottimizzazione database, dimostrando la capacità di sfruttare funzionalità specifiche del DBMS (Full-Text Search).

Limitazioni: Il limite è hard-coded a 20 risultati ottimizza la velocità per l'autocompletamento, ma non supporta una paginazione profonda per ricerche generiche.

3. Chat della Community in Tempo Reale

Soluzione: Per creare un senso di community, avevamo bisogno di un modo per far interagire gli utenti. Abbiamo quindi implementato un Sistema di Chat Persistente tramite socket.io, supportato da MongoDB.

Problematiche affrontate: Il problema era gestire lo stato in tempo reale senza bloccare i thread principali dell'applicazione o rallentare il database del catalogo film con operazioni di scrittura ad alta frequenza.

Funzionamento: All'apertura della Chat Room di un film ogni utente invia un messaggio di connessione attraverso il Socket. In caso siano già presenti messaggi su questa chat room essi verranno caricati e mostrati con la struttura: nome utente, orario di invio e testo del messaggio. Invece all'invio del messaggio il Socket effettuerà un'emit con la stessa struttura.

Questi messaggi vengono istantaneamente salvati come documenti in una collection MongoDB e per poi essere trasmessi e visualizzati nell'interfaccia utente.

Requirements: Questo soddisfa il requisito per la gestione di "Dati Dinamici" e contenuti generati dagli utenti. Aggiungendo una componente interattiva ad alta intensità di scrittura giustifica l'uso del database secondario NoSQL.

Limitazioni: Nonostante sia divisa in chat room con la crescita della community esse potrebbero diventare molto caotiche visto che non è richiesto il login. Tuttavia,

una volta implementato questo, l'architettura è estensibile: lo schema MongoDB può essere facilmente modificato e, in combinazione con un metodo di login sicuro, si potrebbe usufruire di sistemi di monitoraggio delle chat room.

Conclusioni

L'utilizzo di **Vanilla JavaScript** e **Handlebars** per il frontend, **Node.js** per la gestione delle API e dei socket, e **Spring Boot** per la gestione dei dati statici, ci ha permesso di sviluppare una soluzione bilanciata, performante e facilmente scalabile.

L'integrazione di due diversi sistemi database, **PostgreSQL** per la coerenza relazionale e **MongoDB** per la flessibilità dei dati dinamici, ci ha aiutato a gestire la complessità del sistema.

Va sottolineato che le funzionalità descritte rappresentano solo il nucleo centrale di **Movie APP**: la piattaforma è stata progettata in modo modulare, lasciando ampio spazio a future evoluzioni e potenziamenti, come l'affinamento dei sistemi di filtraggio o l'implementazione di algoritmi di ricerca avanzati.

Divisione Lavoro

Mattia: Frontend (stile e display dei dati), Main Server (gestione degli errori e comunicazione con gli altri server), Express Server (comunicazione con Main Server)

Fiorella: SpringBoot Application(JPA, Swagger, JavaDoc) Main Server (socket communication), Express Server (schema, query).

Entrambi però sottolineiamo che abbiamo collaborato in tutte le fasi del progetto in presenza o su discord quando non possibile, raramente da soli.

Informazioni Extra

- **MongoDB**: Abbiamo importato ed utilizzato un dataset pulito e normalizzato per la nostra soluzione, le nostre collections sono state esportate in [questo file](#)
- **PostgreSQL**: Il nostro database viene esportato in [questo file SQL](#), contenente le nostre tabelle, con chiavi primarie ed esterne, proveniente dai dataframe puliti. Eseguire su pgadmin un'operazione psql di import del database: \i {path_SQL_dump_DB}

Bibliografia

- Gemini: Utilizzato principalmente per l'aiuto nella costruzione di componenti moderni e per problem solving di piccola entità.
- Guida alla documentazione java:
<http://www.oracle.com/technetwork/java/javase/documentation/index-137868.html>

- [Specifications Java SPA](#)
- [Mastering Spring Data JPA Specifications for Robust Data Access](#)